results found across iterations, and when it gets stuck, it moves to one of these previous (and advantageous) positions, resetting the descent or just moving to a different area of the problem space.

We have mentioned, if you remember, that despite being lower with respect to greedy algorithms, the probability of getting stuck in local minima is still not null.

In particular, this event can be concurrently caused by a few factors:

- Non-optimal choice of the algorithm's parameters: cooling becomes, for instance, too fast, and the algorithm gets stuck away from global minimum;
- "Bad luck": as we said, it's a stochastic algorithm after all, and it might reach the optimum too soon, and never be able to go back once the system cools down;
- The update step (and more likely so): while we saw that random sampling causes the fewest constraints towards other areas in the problem space with lower energy, it will slow down local convergence (possibly too much to be acceptable);
- Sometimes it's possible to have update rules that allow long steps, while other times (as we'll see) it might be easier to implement only small steps, which means that those long leaps to different areas that could bring the algorithm outside of local minima are more rare.

When some or all of these happen, using random restart could save the day.